Hertie School

Natural Language Processing

Session 4: Text Representation II
Distribution-based approaches

GRAD-E1282 · Fall Semester 2026
Luis Fernando Ramírez Ruiz · l.ruiz@hertie-school.org

Session 3: dimensions we assigned

TOKENS → VOCABULARY → WEIGHTS → VECTOR

CHOICE 1 · FEATURES

What does a column stand for?

unigrams · n-grams · characters

CHOICE 2 · WEIGHTS

What number goes in the cell?

binary · counts · TF-IDF

COMPARISON · COSINE

How close are two rows?

direction, not length: 0 to 1

WHERE COUNT VECTORS FALL SHORT

01 Related words look unrelated

"dog" and "cat" are separate columns: nothing says they are similar.

02 Huge, mostly empty vectors

One dimension per vocabulary item, almost every cell zero.

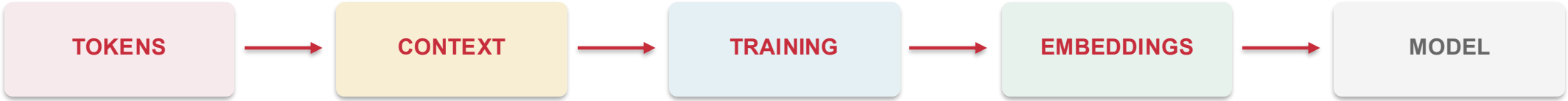
03 Unseen words have no place

A word not seen during fitting gets no dimension at all.

Today: learn a few hundred dense dimensions from context, instead of assigning one per word.

Not solved today: word order. Embeddings do not restore it either — that waits for Sessions 8–9.

Today: from counting to learning



By the end of today you should be able to:

- Explain the distributional hypothesis and distinguish distributional from distributed representations.
- Explain the Word2vec algorithm, then how CBOW and SkipGram learn an embedding matrix.
- Use pre-trained embeddings, and aggregate word vectors into text vectors.
- Choose between fastText, Doc2vec and pre-trained models — and tell static from contextualized embeddings.

Core Python tools

gensim · fastText · spaCy

The lab will train embeddings on a small corpus and inspect their nearest neighbours.

From similarity to hypothesis: the logical chain

01 DISTRIBUTIONAL SIMILARITY

A word's meaning is read off the words around it. "NLP rocks": literally stones, but the context makes it mean great.

02 DISTRIBUTIONAL HYPOTHESIS

So two words appearing in similar contexts must have similar meanings — like "dog" and "cat".

03 WHAT THIS BUYS US

In a vector space the vector is the meaning, so similar words must have vectors close together.

Step 02 in practice

The ____ barked at the postman.	**dog**
She fed the ____ this morning.	**dog / cat**
Our ____ sleeps on the sofa.	**dog / cat**
The ____ chased a mouse.	**cat**

Two of these slots take either word. That shared distribution — not the letters d-o-g — is the only evidence Word2vec ever sees that the two are related.

Distributional vs. distributed: same evidence, two vector spaces

DISTRIBUTIONAL · count the contexts

	barked	fed	sofa	mouse	engine	…
Doc 1	3	2	2	0	0	0
Doc 2	0	2	3	2	0	0
Doc 3	0	0	0	0	4	0

|V| dimensions — one per vocabulary word
sparse — most cells are 0
named — you can read every column
one-hot · BoW · n-grams · TF-IDF

compress
|V| → d

DISTRIBUTED · learn d numbers

	d1	d2	d3	d4	…	d300
Doc 1	.61	-.20	.33	.05	…	.12
Doc 2	.58	-.12	.41	.02	…	.09
Doc 3	-.30	.72	-.05	.44	…	-.21

d dimensions — typically 50–500
dense — almost no zeros
unnamed — dog and cat simply end up close
Word2vec · GloVe · fastText · Doc2vec

Same evidence, different storage: both record which words share contexts — as |V| raw counts, or as d learned coordinates.

Embedding: the mapping from the distributional space into the distributed space, for the words of a corpus.

Word embeddings: what we gain

King − Man + Woman ≈ Queen

LOW-DIMENSIONAL

Vectors of 50–500 dimensions instead of the several thousand produced by TF-IDF on a realistic corpus.

DENSE

Most values are non-zero. Downstream machine learning tasks become more tractable and more efficient.

LEARNED FROM CONTEXT

For every word w the vector is refined by prediction against the words in its context C, using a two-layer neural network.

The training task is a pretext. We do not care about predicting words — we want the weight matrix the network learns along the way.

Word2vec: the algorithm in four steps

1 INITIALISE

Give every vocabulary word a random vector of d numbers — one row of the matrix E.

→

2 SLIDE A WINDOW

Move a window of 2k+1 words over the corpus: one centre word, k neighbours on each side.

→

3 PREDICT

Use the current vectors to predict the centre word from its neighbours — or the reverse.

→

4 UPDATE

Compare with the words actually there; backpropagation nudges the vectors toward the right answer.

repeat for every window, over several passes (epochs)

WHY IT NEEDS NO LABELS

The “labels” are the neighbouring words themselves. Any raw text is training data — no dictionary, no annotation.

TWO WAYS TO POSE STEP 3

`CBOW` neighbours → centre word

`SkipGram` centre word → each neighbour

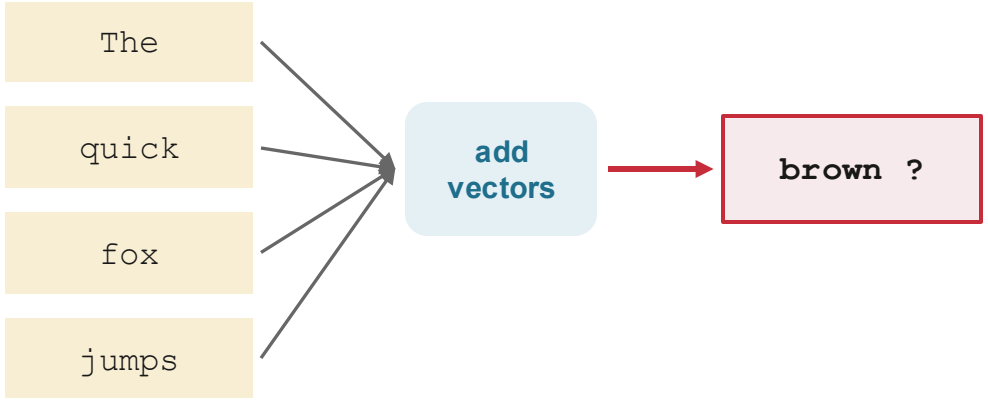
Words that keep appearing with the same neighbours get pushed toward the same vectors.

CBOW: predict the centre word from its context

The	quick	**brown**	fox	jumps	over	the	lazy	dog

■ centre word
■ context, k = 2

ONE WINDOW, ONE TRAINING EXAMPLE



X = the 2k neighbours **Y** = the centre word

SLIDING THE WINDOW: ONE EXAMPLE PER POSITION

```
((quick, brown), The)
((The, brown, fox), quick)
((The, quick, fox, jumps), brown)
((quick, brown, jumps, over), fox)
...
```

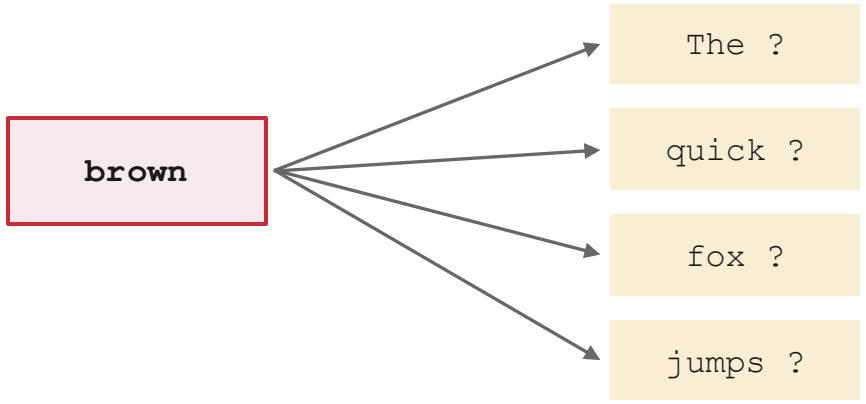
CBOW asks: given the neighbours, which word is missing? many → one

SkipGram: predict the context from the centre word

The	quick	**brown**	fox	jumps	over	the	lazy	dog

■ **centre word**
■ **context, k = 2**

ONE WINDOW, 2k TRAINING PAIRS



X = the centre word **Y** = each neighbour, one at a time

THE SAME WINDOW AS (X, Y) PAIRS

(brown, The)

(brown, quick)

(brown, fox)

(brown, jumps)

4 pairs

from one window (CBOW: 1 example)

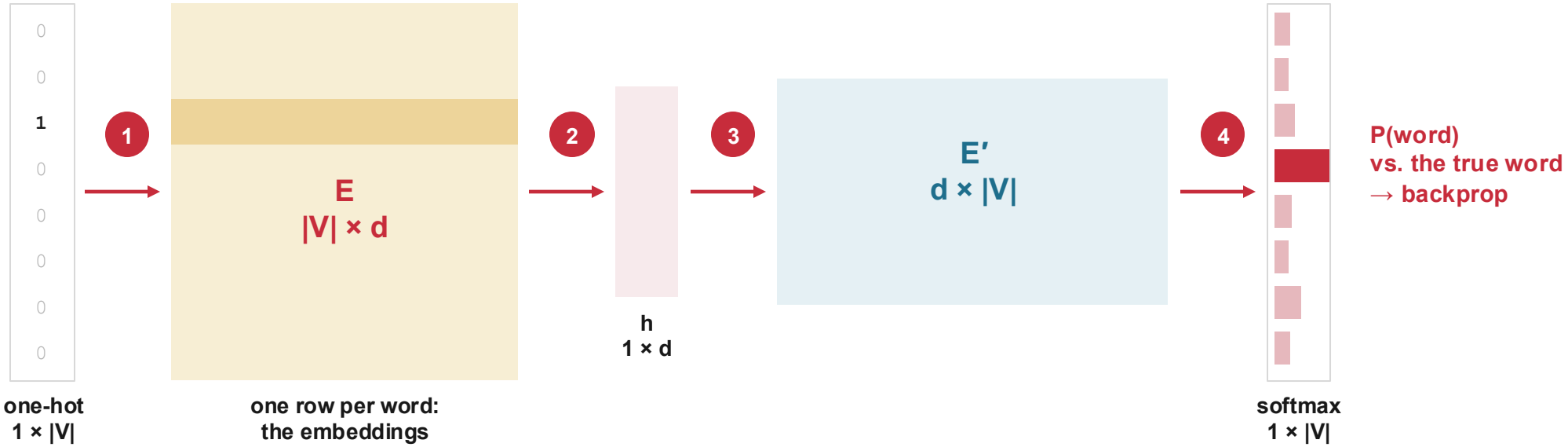
→ **better for small corpora and rare words; slower to train**

SkipGram asks: given this word, which neighbours would you expect? one → many

Session Break

When we return: inside the Word2vec network, one training step with numbers, and embeddings in practice.

Inside the network: the embedding matrix is a lookup table



01 INPUT

Word indices fetch their rows of E, initialised randomly.

02 SUM

CBOW adds the 2k context rows; SkipGram uses the one centre row.

03 PROJECTION

Multiply by E′ (d × |V|): one score per vocabulary word.

04 SOFTMAX

Probabilities over the vocabulary; compared with the label, backprop updates E and E′.

After training, keep E: row i is the vector for word i. Next slide: the same steps with numbers.

One training step, with numbers (CBOW)

1 · LOOK UP

rows of E

The	0.1	0.2	-0.2
quick	0.0	0.4	0.4
fox	-0.1	-0.5	0.0
jumps	0.1	-0.2	0.2

Each context word's index selects its row of E (|V| × d).

2 · COMBINE

h = sum

h =

0.1	**-0.1**	**0.4**

Add the 4 rows: one d-vector for the whole context.

3 · SCORE

h × E′

the	-0.08
quick	0.00
brown	-0.09
fox	-0.07
jumps	-0.03
over	0.19
lazy	0.10
dog	-0.03

E′ (d × |V|) holds one column per word: score = fit between h and that word.

4 · SOFTMAX

probabilities

the	0.12
quick	0.12
brown	**0.11**
fox	0.12
jumps	0.12
over	0.15
lazy	0.14
dog	0.12

Exponentiate and normalise: sums to 1. Untrained, so almost uniform.

5 · LEARN

update

true word: **brown**

loss = −log 0.11

= 2.17

Backpropagation moves the 4 context rows of E and brown's column of E′ so that P(brown) rises next time.

Repeat for every window, over many epochs: words that share contexts get pushed the same way.

After training, keep E: its rows are the word embeddings. E′ only served the prediction.

Pre-trained embeddings: always start here

Word2vec
Google

GloVe
Stanford

fastText
Facebook AI

Available for d = 25, 50, 100, 200, 300, 600

A PRE-TRAINED MODEL IS A LOOKUP TABLE

beautiful	→	[-0.018, 0.056, -0.012, 0.073, …]
gorgeous	→	[0.041, 0.022, -0.087, 0.105, …]
…	→	3 million words × 300 dimensions

Nearest neighbours of “beautiful” (cosine)

gorgeous	0.835
lovely	0.811
stunningly_beautiful	0.733
wonderful	0.685
magnificent	0.659

Check membership first. Querying a word outside the model vocabulary raises a “key not found” error — the OOV problem has not gone away, it has only moved.

From word vectors to text vectors

In most applications we deal with sentences, paragraphs or full documents — not isolated words.

The simple approach: sum or average

Break the text into words, take the embedding of each, and combine them. This may not capture aspects of the text as a whole but it works surprisingly well in practice.

```
import spacy
nlp = spacy.load("en_core_web_sm")

doc = nlp("Canada is a large country")

doc[0].vector   # vector for 'Canada'
doc.vector      # averaged sentence vector
```

The OOV problem, again

- Exclude OOV words from feature extraction, acceptable if the model was trained on a large corpus.

< 80%

If the overlap between corpus vocabulary and embedding vocabulary falls below 80%, expect poor performance. Vocabulary overlap is a cheap heuristic for gauging fit before you train anything.

fastText: embeddings built from character n-grams

A different response to OOV: change the training process to bring in subword information.

- A word can be represented by its constituent character n-grams — morphological information such as prefixes, suffixes and word endings.
- fastText learns embeddings for words and character n-grams together, following an architecture similar to Word2vec.
- A word's embedding is an aggregation of its character n-gram embeddings.
- This makes it possible to generate an embedding even for a word absent from the vocabulary.

"gregarious" — not in the vocabulary

gre	reg	ega	gar	…	ous

Combine the n-gram embeddings and you obtain an embedding for "gregarious", even though the whole word was never seen.

Where each representation stands on OOV

TF-IDF
no vector at all

Word2vec
no vector; drop it or randomise

fastText
vector built from character n-grams

Doc2vec: representations for texts of arbitrary length

Averaging word vectors ignores the context of words in the text. Doc2vec learns the text representation directly.

The problem: “dog bites man” and “man bites dog” receive the same representation under both Word2vec and fastText — the same trap we saw with Bag of Words in Session 3.

- Based on the paragraph vectors framework; implemented in gensim.
- Alongside the word vectors it learns a paragraph vector representing the full text, words in context included.
- Paragraph vectors are unique to a given text; word vectors are shared across all texts.
- Vectors for new texts are inferred from the word vectors learned during training.

DM — Distributed Memory

The paragraph id and the word vectors are averaged or concatenated to predict the next word. Mirrors CBOW.

DBOW — Distributed Bag of Words

The paragraph id alone predicts words sampled from the text. Mirrors SkipGram.

One idea, four levels of granularity

Everything today learns dense vectors from context. The methods differ in what gets a vector.

CHARACTER n-GRAMS

fastText

Vectors for subword units, aggregated into word vectors. Handles words never seen in training.

WORDS

Word2vec · GloVe

One vector per vocabulary word, learned by predicting words from context. No OOV fallback.

TEXTS

Doc2vec

One vector per sentence, paragraph or document, learned with the words in context.

AGGREGATES

sum · average

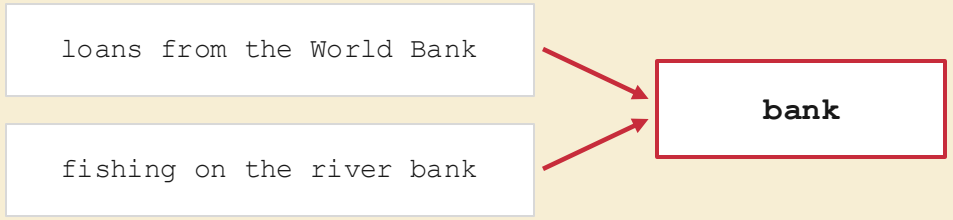
No new training: combine existing word vectors. Simple, fast, order-blind.

Still true of all four: a word has exactly one vector, whatever sentence it appears in. That is what “static” means — next slide.

Static vs. contextualized embeddings

STATIC

Word2vec · GloVe · fastText

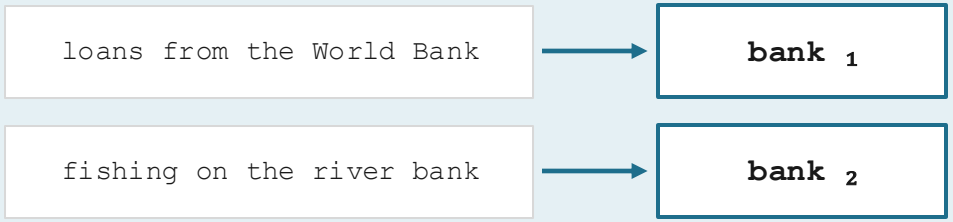


One vector per word type. Context shapes it only during training; afterwards it is a fixed lookup.

All senses blended: finance and river share one vector.

CONTEXTUALIZED

BERT and other Transformers · Sessions 8–9



One vector per word token. Computed from the words before and after, every time; order matters.

Senses separate: “not happy” lands near “unhappy”.

Static: a lookup table. **Contextualized:** a function of the whole sentence.

Everything today is static, which is right for analogies, word-level reasoning and cheap features; blind to word sense.

Choosing a representation for your research note

Small corpus, interpretability matters, you need to name the features driving a result	**TF-IDF**
You need similarity between different words, and a general-domain corpus in a major language	**pre-trained Word2vec or GloVe**
Morphologically rich language, noisy text, many misspellings or rare technical terms	**fastText**
You need one vector per document and word order carries part of the signal	**Doc2vec**
Your domain vocabulary barely overlaps with any pre-trained model	**train your own embeddings**

Quick check: which embedding would you start with?

Choose a starting representation - and say which limitation you are accepting in exchange.

A	Classify German municipal reports full of long compound nouns and rare technical terms.	**fastText?**
B	Cluster 40,000 news articles by topic, one vector per article.	**Doc2vec?**
C	Find which words a corpus treats as interchangeable with “migration”.	**Word2vec?**

2 min in pairs → 2 min discussion.

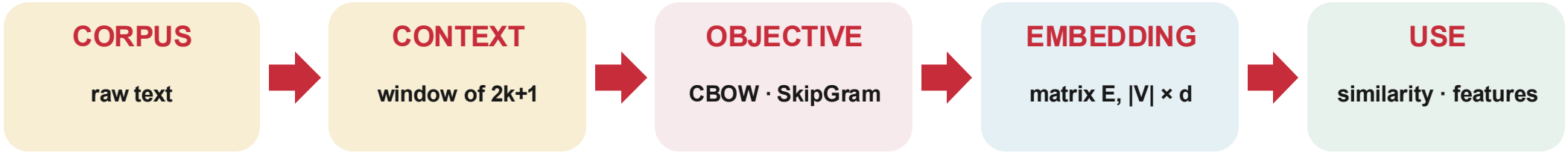
Distributed representations, and what they still cannot do

01	**ONE VECTOR PER WORD TYPE**	“Bank” by a river and “bank” with a vault share a single vector. Representations are static, not contextual.
02	**WORD ORDER STILL LOST**	Summing or averaging word vectors gives “dog bites man” and “man bites dog” the same representation.
03	**CORPUS BIAS IS INHERITED**	Vectors reproduce the associations of the training corpus, including social bias. Pre-trained does not mean neutral.
04	**OPAQUE AND HARD TO EVALUATE**	Dimensions carry no interpretable meaning, and quality is only measurable on a downstream task.

NEXT: SUPERVISED MODELS, THEN CONTEXT

Session 5 uses these features for classification. Sessions 8–9 replace static vectors with contextual ones from Transformers.

Keep this mental model



THE RECURRING QUESTION

What information does this representation make visible to the model — and what does it hide?

TODAY'S HIERARCHY

Word2vec → fastText → Doc2vec

all static — contextual vectors in Sessions 8–9

Visible now: similarity between different words, dense features. **Still hidden:** word order, word sense, what each dimension means.

Next session: use these features in a supervised model — text classification with logistic regression.

Where we are in the NLP pipeline



Data acquisition	Text extraction, normalization	Representation / features	Basic models	Neural models	Evaluation	Final evaluation + XAI	Deployment / orchestration
Not covered	Session 2	SESSIONS 3–4	Sessions 5 & 7	Sessions 6, 8-9	Throughout	Session 10	Sessions 11-12

gensim fastText spaCy · next: supervised models for text classification

Today: learn dense representations that encode similarity. Next: train models on top of them.